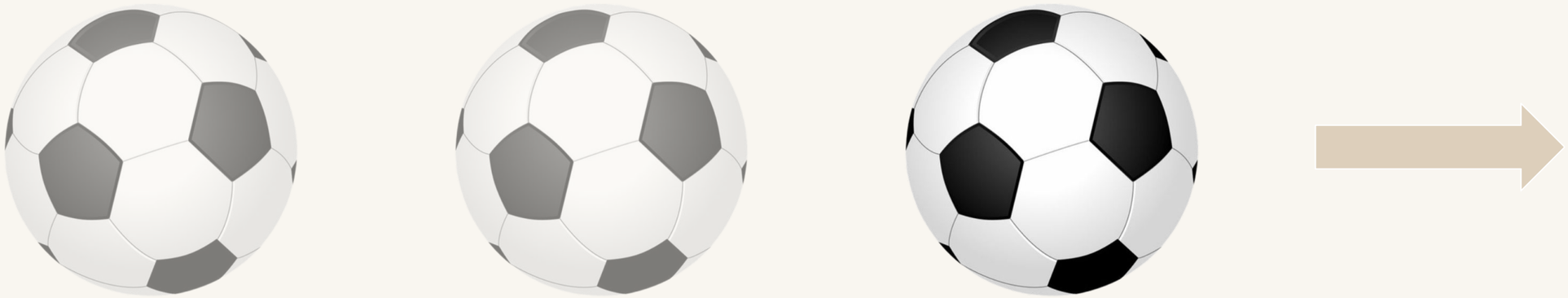
Sequence Modeling

-PLN

Given an image of a ball, can you predict where it will go next?

Given an image of a ball, can you predict where it will go next?

Given an image of a ball, can you predict where it will go next?



Sequences Matters

Application of sequence modeling

- Sentiment classification
- Music generation
- Next word prediction

.....

Predict the next word

Today my cat was sitting on a mat

Predict the next word

Today my cat was sitting on a mat

given these words

Predict the next word

Today my cat was sitting on a mat

given these words

predict
next word

Preprocess Idea 1: use a fixed window

Today my cat was sitting on a mat

given these **two** words

predict next word

Dictionary(10k) = {
a
alice
arone
….
….
….
zolo

<eos>
<unk>
}

One-hot feature encoding: tells us what each word is

on - [..1 0 0 0 0..] // size of 10k vector
a - [1 0 0 0……]

Problem 1: can’t model long-term dependencies

My name is **pollock**. I love to play **cricket** but my brother **rupok** loves to play **football**. Who loves to play cricket _____ ?

We need information from the distant past to accurately predict the correct word

Preprocess Idea 2: use a really big fixed window

Today my cat was sitting on a mat

given these **seven** words

predict next word

Dictionary(10k) = {
a
alice
arone
….
….
….
zolo

<eos>
<unk>
}

One-hot feature encoding: tells us what each word is

today - [..0 1 0 0 0..]
my - [..0 0 0 0 1 0..]
on - [..1 0 0 0 0..]
a - [1 0 0 0…..]
……..

Problem 2: No parameter sharing

Today my cat was sitting on a mat

given
these
seven
words

predict
next
word

Each of these inputs has separate parameters.

One-hot feature encoding: tells us what each word is
today - [..0 1 0 0 0..]
my - [..0 0 0 0 1 0..]
on - [..1 0 0 0 0..]
a - [..0 1 0 0 0..]
……..

Things we learn about the **sequence won't transfer** if they appear elsewhere in the sequence.

Problem 2: No parameter sharing (Example)

Imagine you are training a model to recognize that the word "cat" refers to an animal.

Scenario A: The model sees "The cat sat." It learns that the 2nd word ("cat") is an animal using one set of weights.

Scenario B: Later, the model sees "Cat is sleeping." Because the word "cat" is now in the 1st position, it uses a different set of weights.

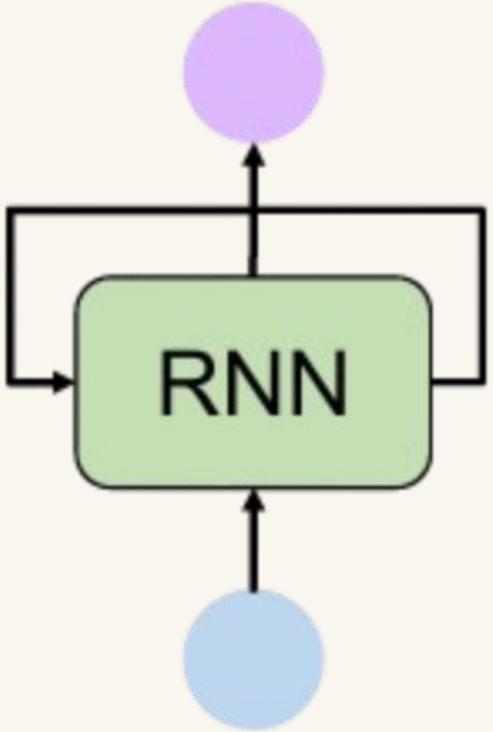
Even though it’s the same word, the model doesn't "transfer" its knowledge from Scenario A to Scenario B. It treats the "cat" at the start of the sentence as something entirely unrelated to the "cat" in the middle.

Recurrent Neural Networks (RNNs) solve this by applying the same weights (parameters) across all steps of the sequence, allowing the model to recognize "cat" no matter where it pops up.

RNN

To model sequences, we need to:

1. Handle **variable-length** sequences (Movie review, machine translation etc.)
2. Track **long-term** dependencies
3. **Share parameters** across the sequence

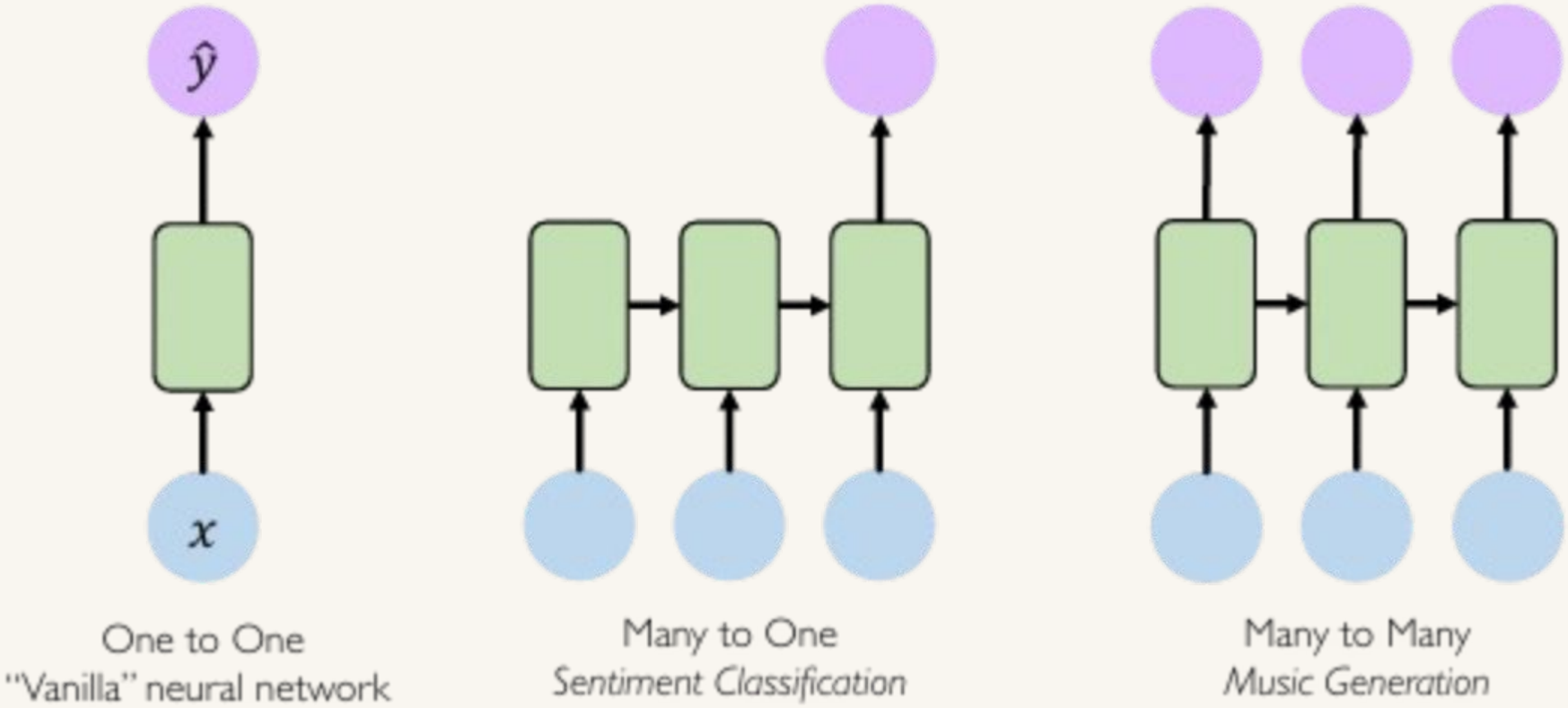


Recurrent Neural Networks (RNNs)

Standard feed-forward neural network

One to One
“Vanilla” neural network

Recurrent neural networks: sequence modeling

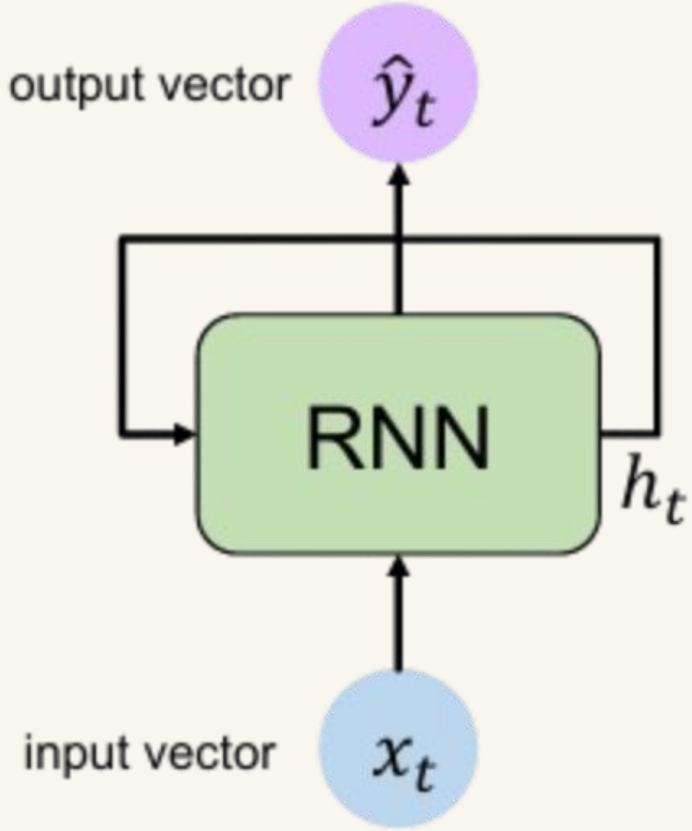


Many more…

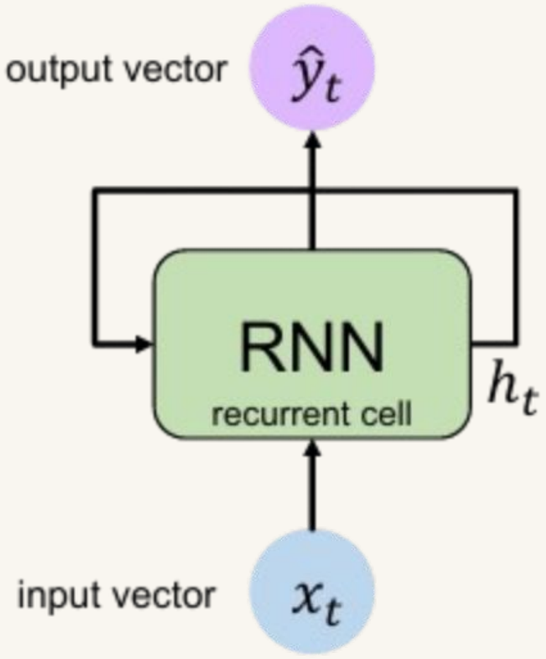
A standard “vanilla” neural network

One to One
“Vanilla” neural network

VS

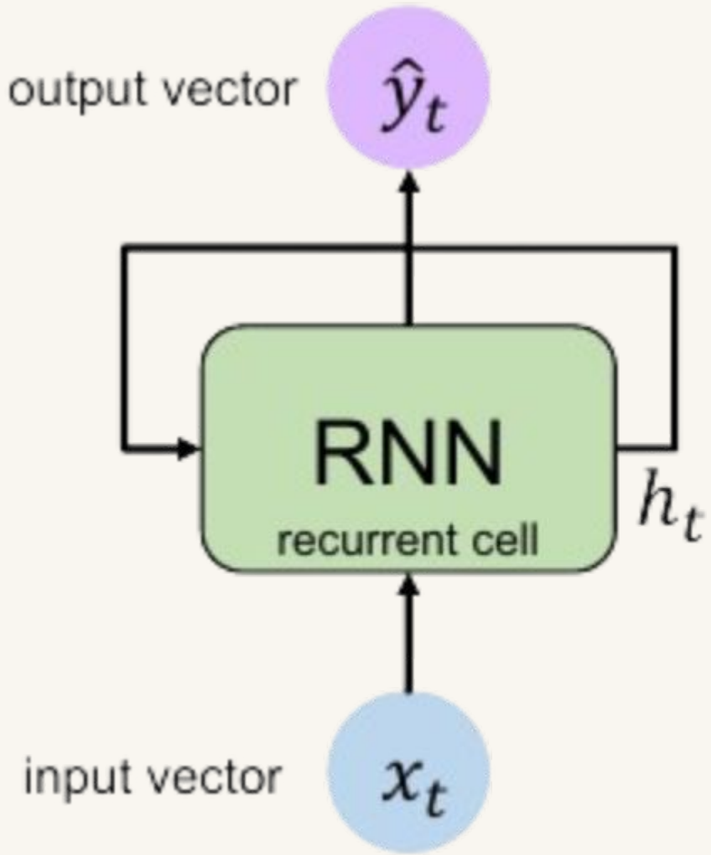


A recurrent neural network (RNN)



Apply a **recurrence relation** at every time step to process a sequence:

A recurrent neural network (RNN)



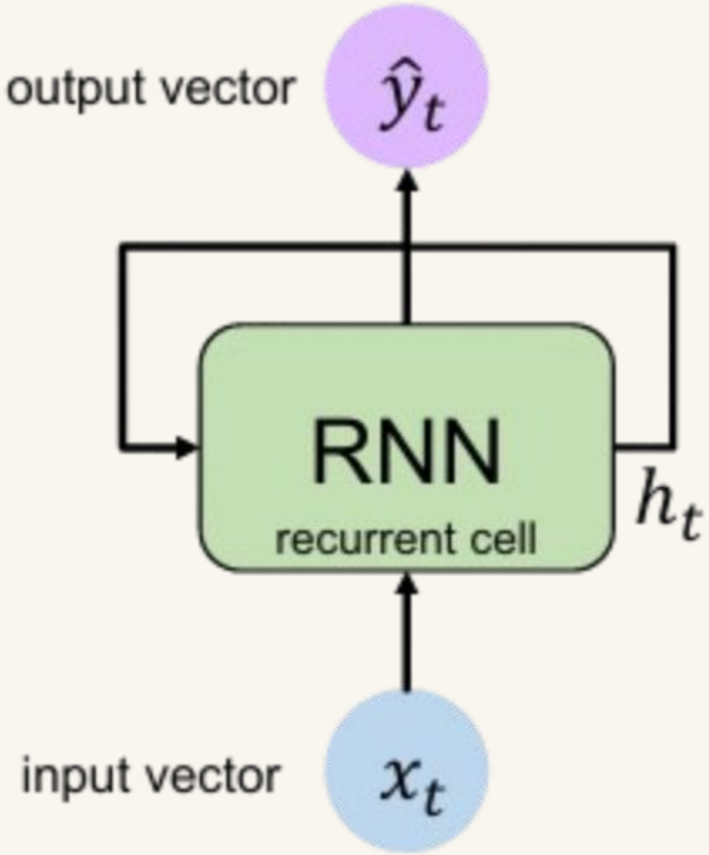
Apply a **recurrence relation** at every time step to process a sequence:

$$\boxed{h_t} = \boxed{f_W}(\boxed{h_{t-1}}, \boxed{x_t})$$

new state — function parameterized by W — old state — input vector at time step t

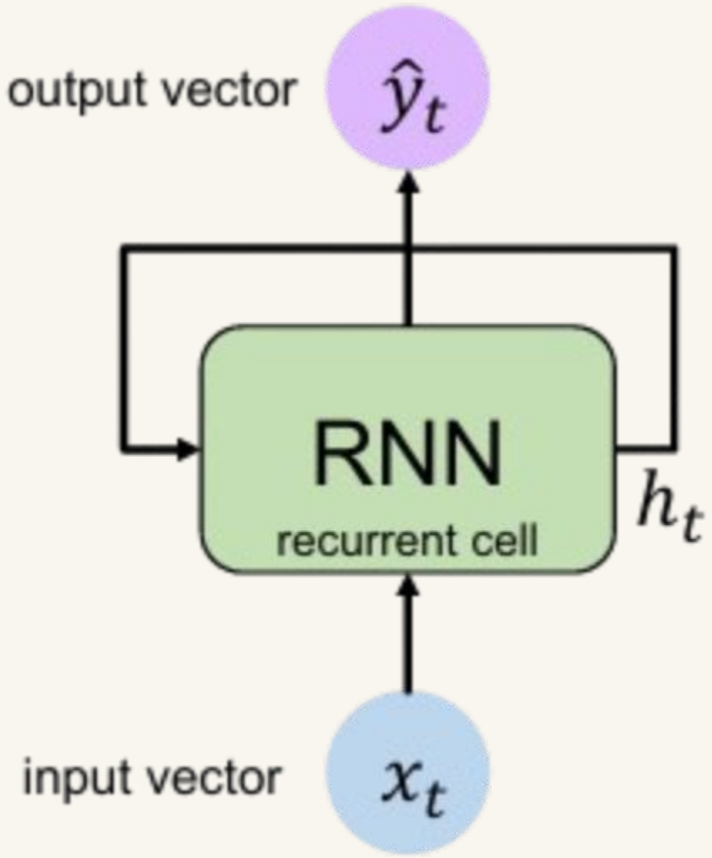
Note: the same function and set of parameters are used at every time step

RNN state update and output



Input Vector

RNN state update and output

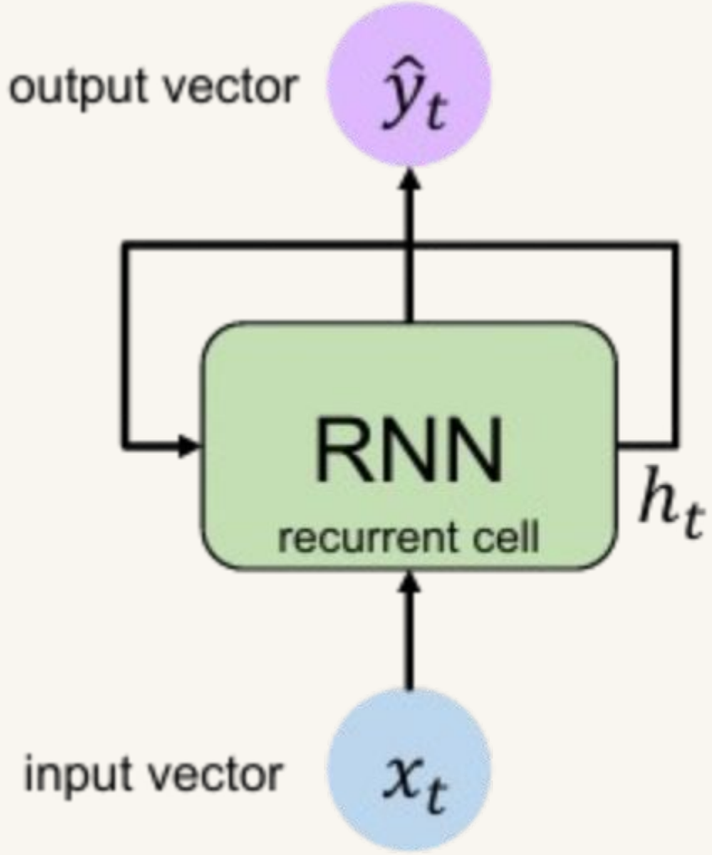


Update Hidden State

$$h_t = \tanh(\boldsymbol{W_{hh}} h_{t-1} + \boldsymbol{W_{xh}} x_t)$$

Input Vector

RNN state update and output



Output Vector

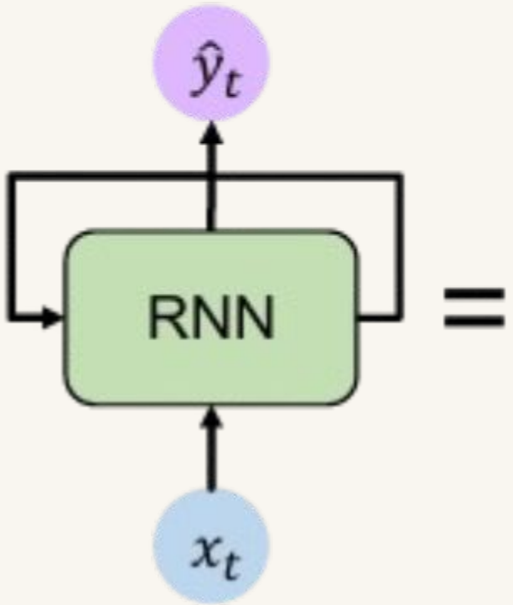
$$\hat{y}_t = \boldsymbol{W}_{\boldsymbol{hy}} h_t$$

Update Hidden State

$$h_t = \tanh(\boldsymbol{W}_{\boldsymbol{hh}} h_{t-1} + \boldsymbol{W}_{\boldsymbol{xh}} x_t)$$

Input Vector

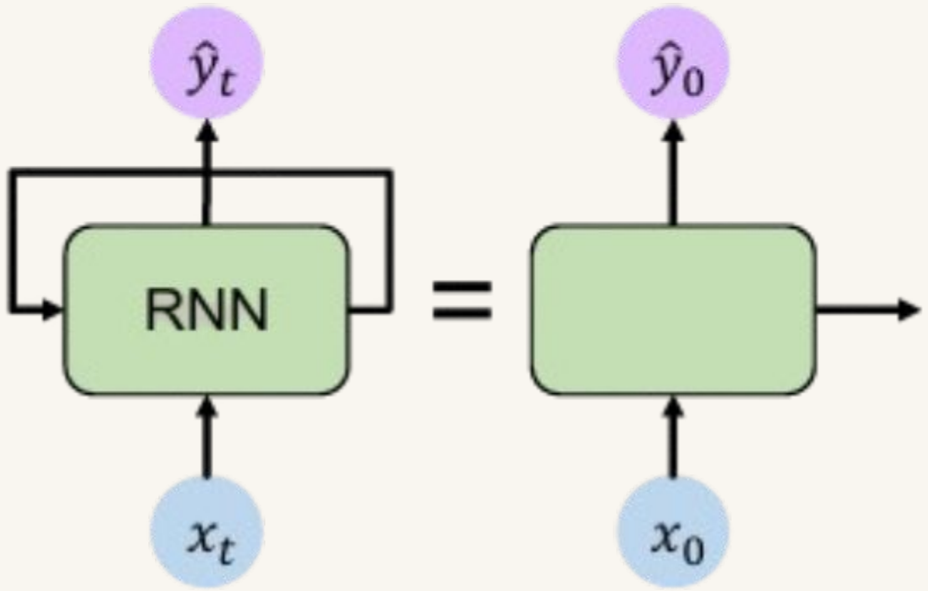
RNNs: computational graph across time



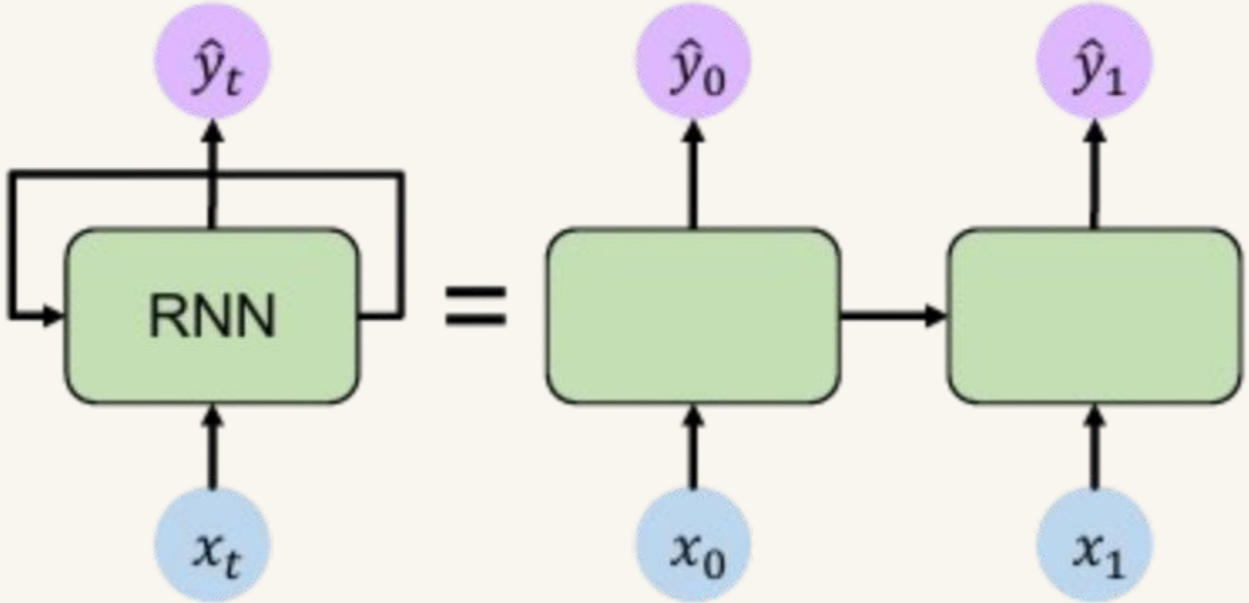
=

Represent as computational graph unrolled across time

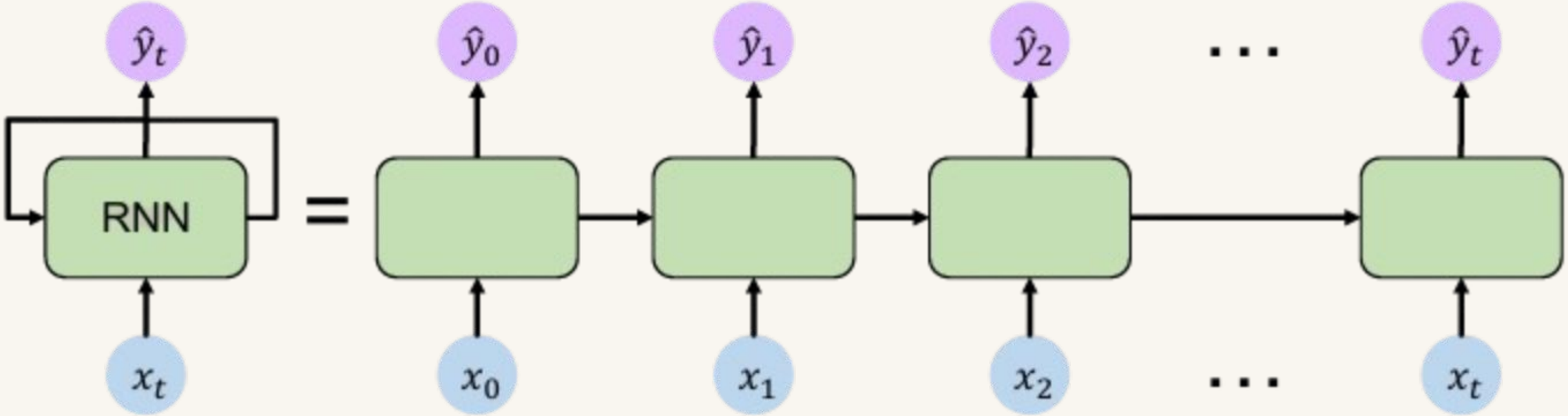
RNNs: computational graph across time



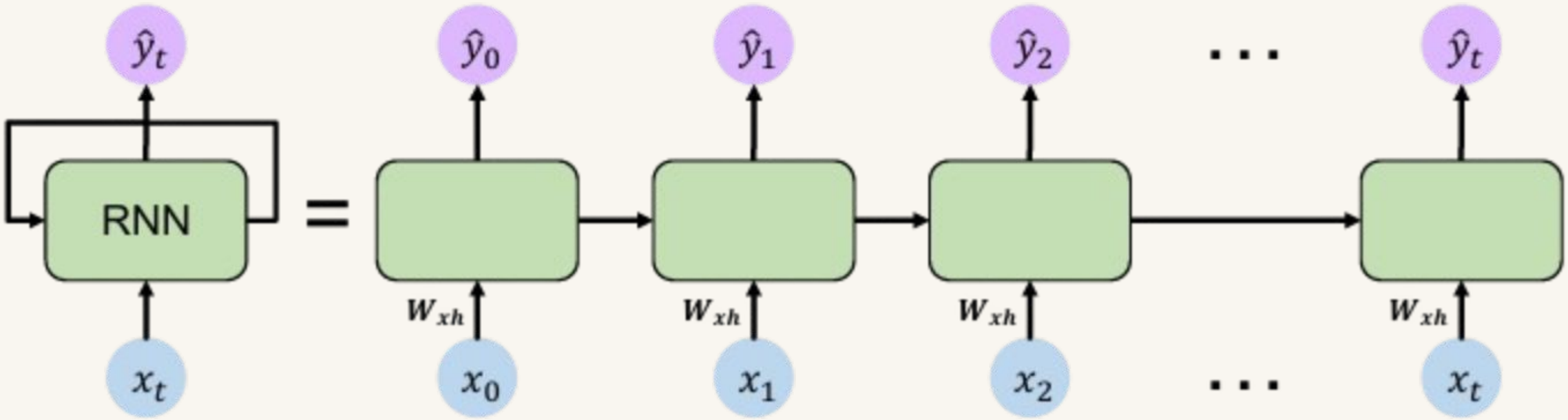
RNNs: computational graph across time



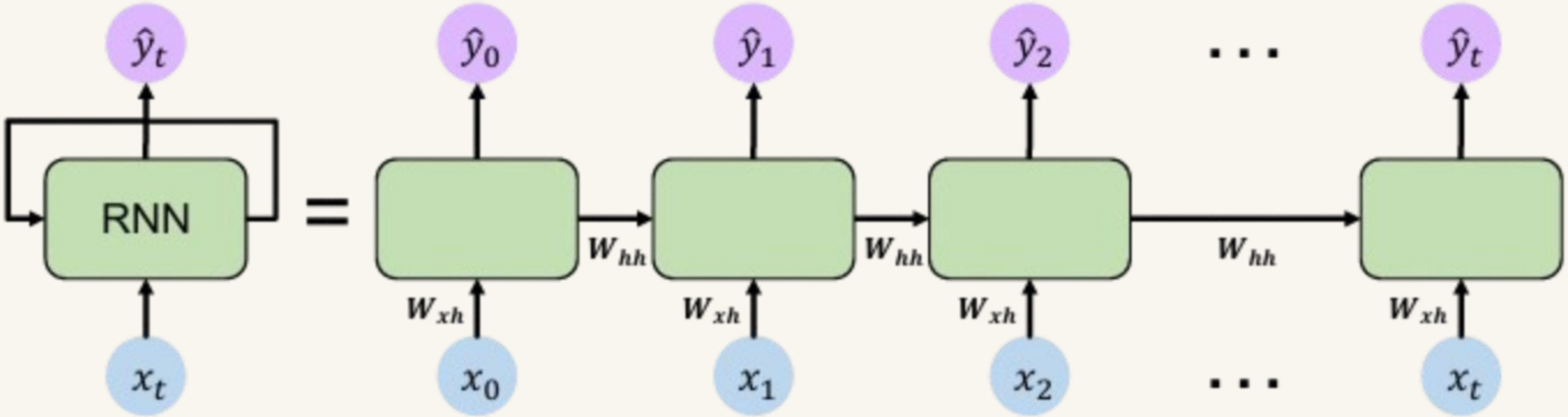
RNNs: computational graph across time



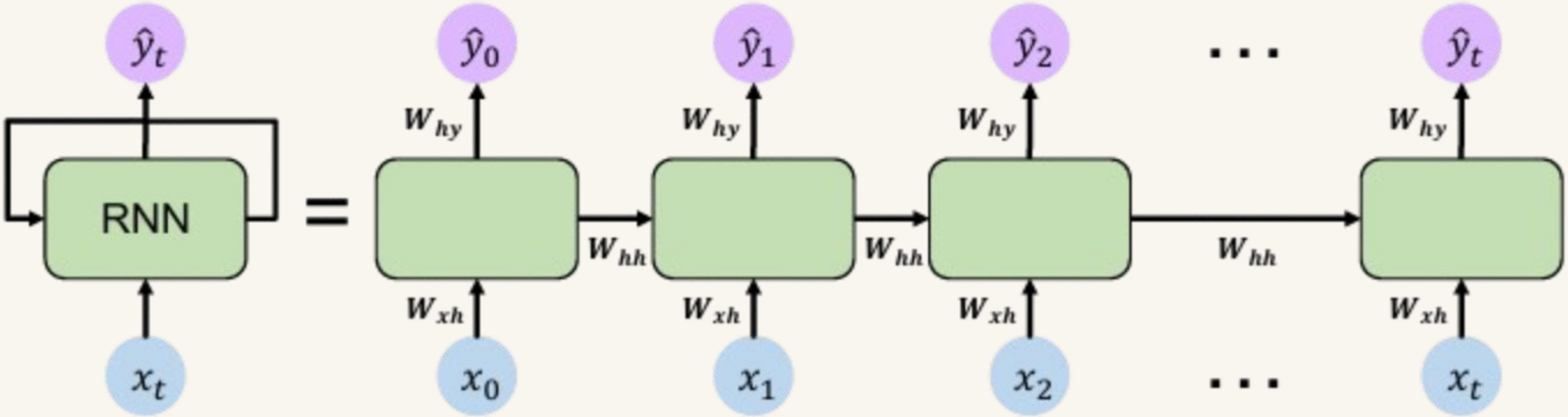
RNNs: computational graph across time



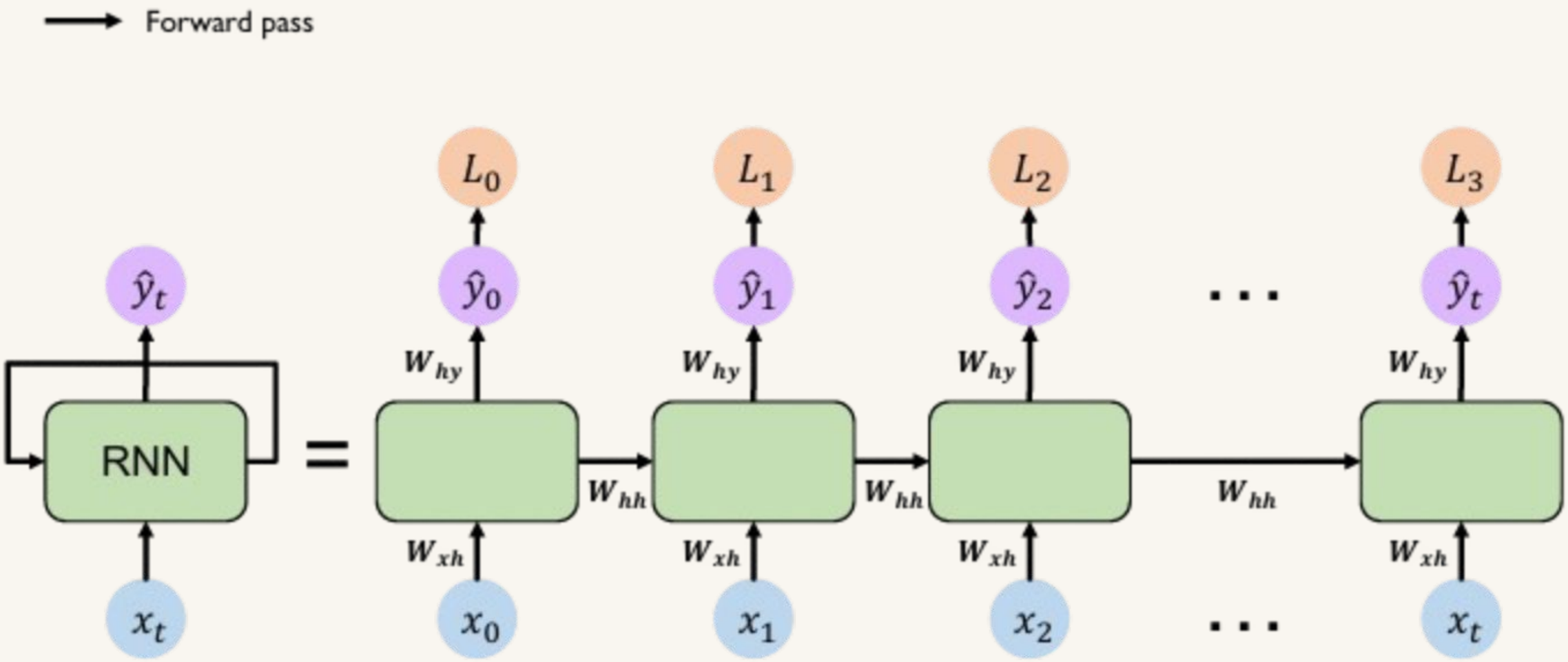
RNNs: computational graph across time



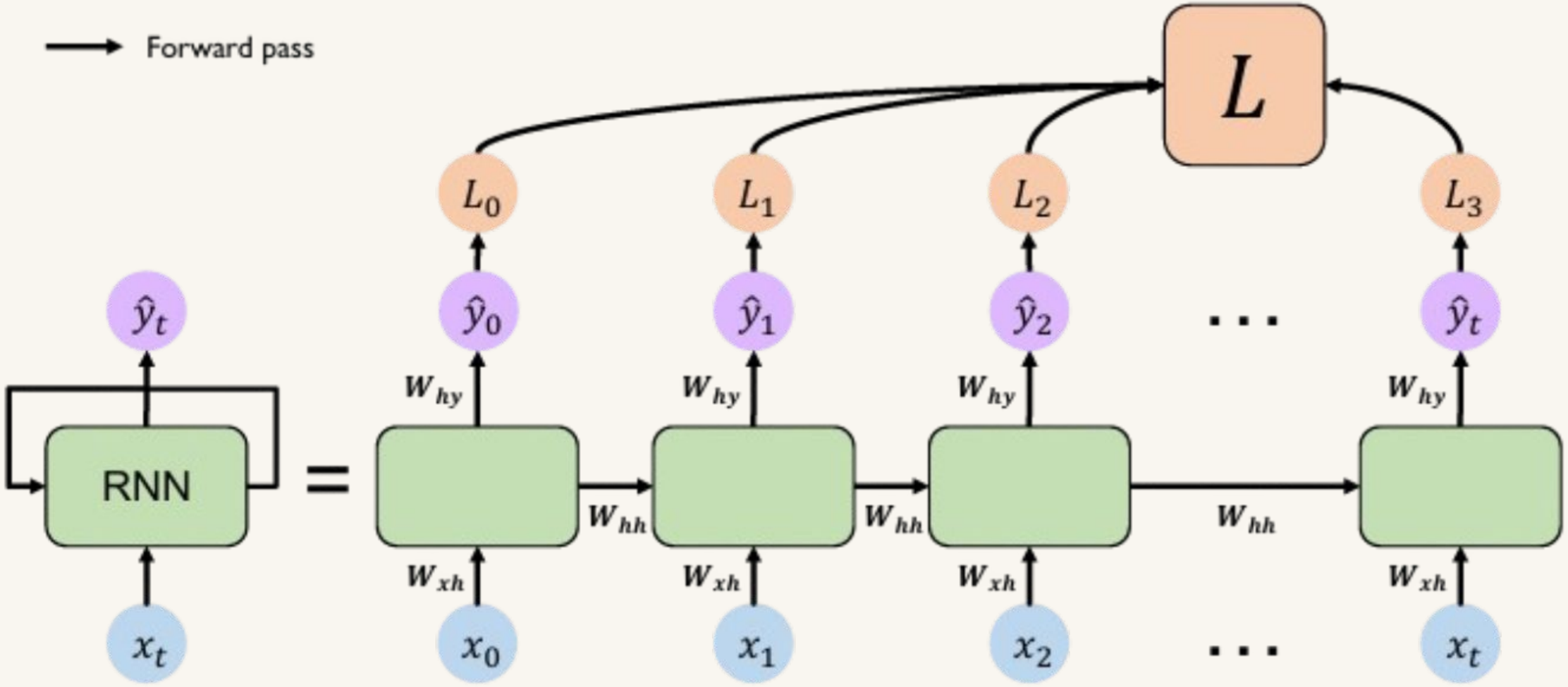
RNNs: computational graph across time



RNNs: computational graph across time

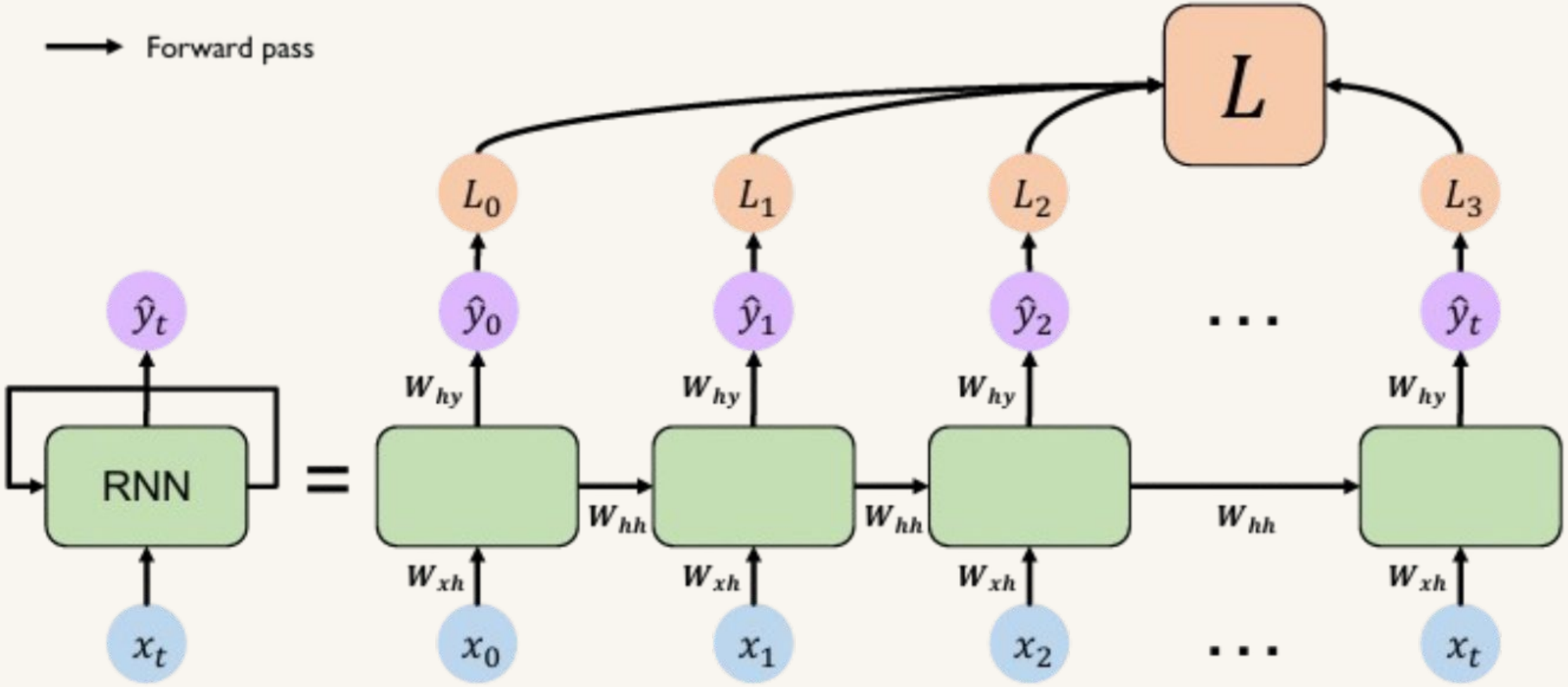


RNNs: computational graph across time



Backpropagation Through Time (BPTT)

RNNs: backpropagation through time



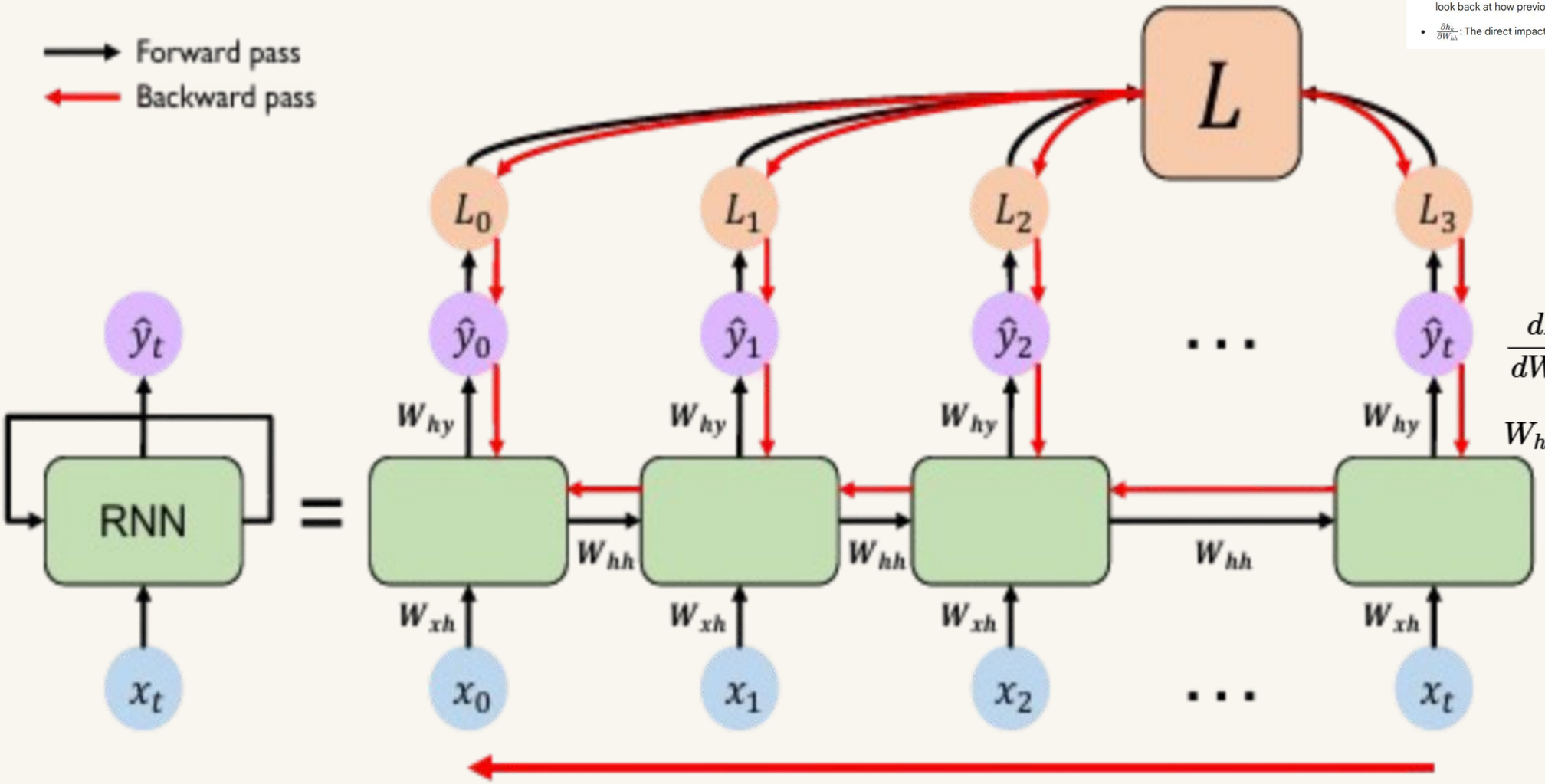
RNNs: backpropagation through time

$$\frac{\partial L}{\partial W_{hh}} = \sum_{k=1}^{t} \frac{\partial L}{\partial \hat{y}_t} \cdot \frac{\partial \hat{y}_t}{\partial h_t} \cdot \left(\prod_{j=k+1}^{t} \frac{\partial h_j}{\partial h_{j-1}} \right) \cdot \frac{\partial h_k}{\partial W_{hh}}$$

Breakdown of the Chain Rule

To make this more digestible, here is what each part represents:

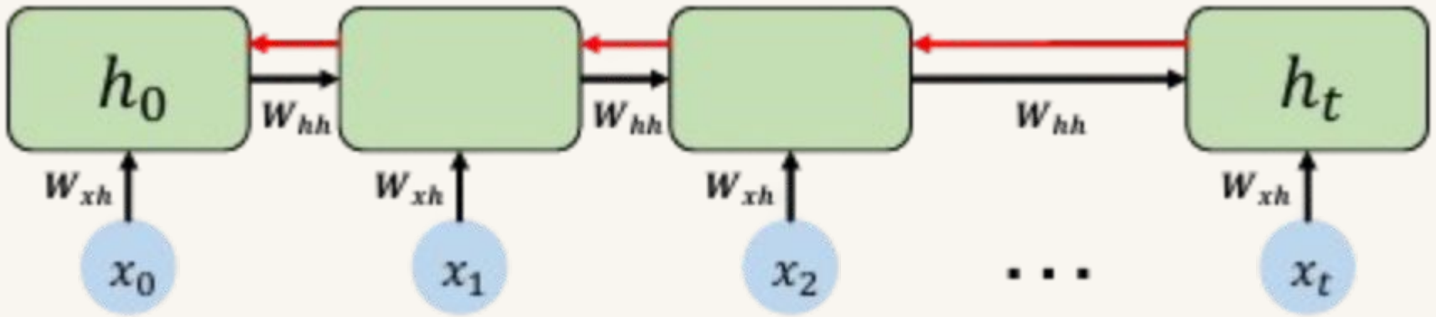
- $\frac{\partial L}{\partial \hat{y}_t}$: How the loss changes with respect to the output.
- $\frac{\partial \hat{y}_t}{\partial h_t}$: How the output changes with respect to the current hidden state.
- $\prod_{j=k+1}^{t} \frac{\partial h_j}{\partial h_{j-1}}$: This is the "temporal" chain. Since W_{hh} is used at every step, we have to look back at how previous hidden states contributed to the current state.
- $\frac{\partial h_k}{\partial W_{hh}}$: The direct impact of the weights on the hidden state at a specific time step k.



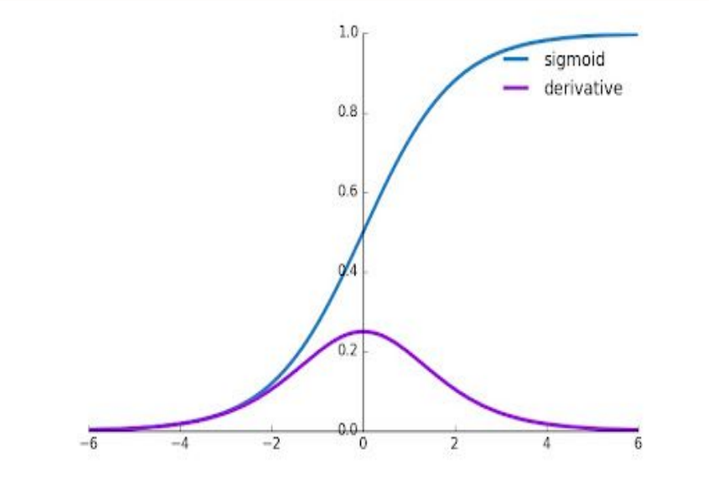
$$\frac{dL}{dW_{hy}} = \frac{dL}{dy_t} \cdot \frac{dy_t}{dW_{hy}}$$

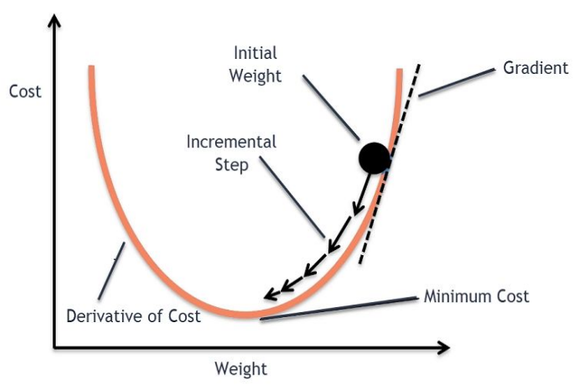
$$W_{hy} = W_{hy} - \eta \cdot \frac{dL}{dW_{hy}}$$

Standard RNN gradient cons



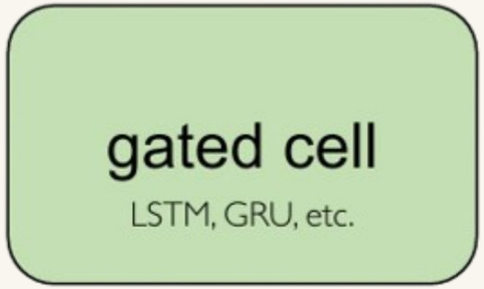
1. Vanishing gradients (Sigmoid)
    i. Running BP for so many times.
    ii. Have smaller and smaller gradients
2. Exploding gradients (If derivative is greater than one)
3. **Bias network to capture short-term dependencies**





Solutions

Use a more **complex recurrent unit** with **gates** to control what information is passed through

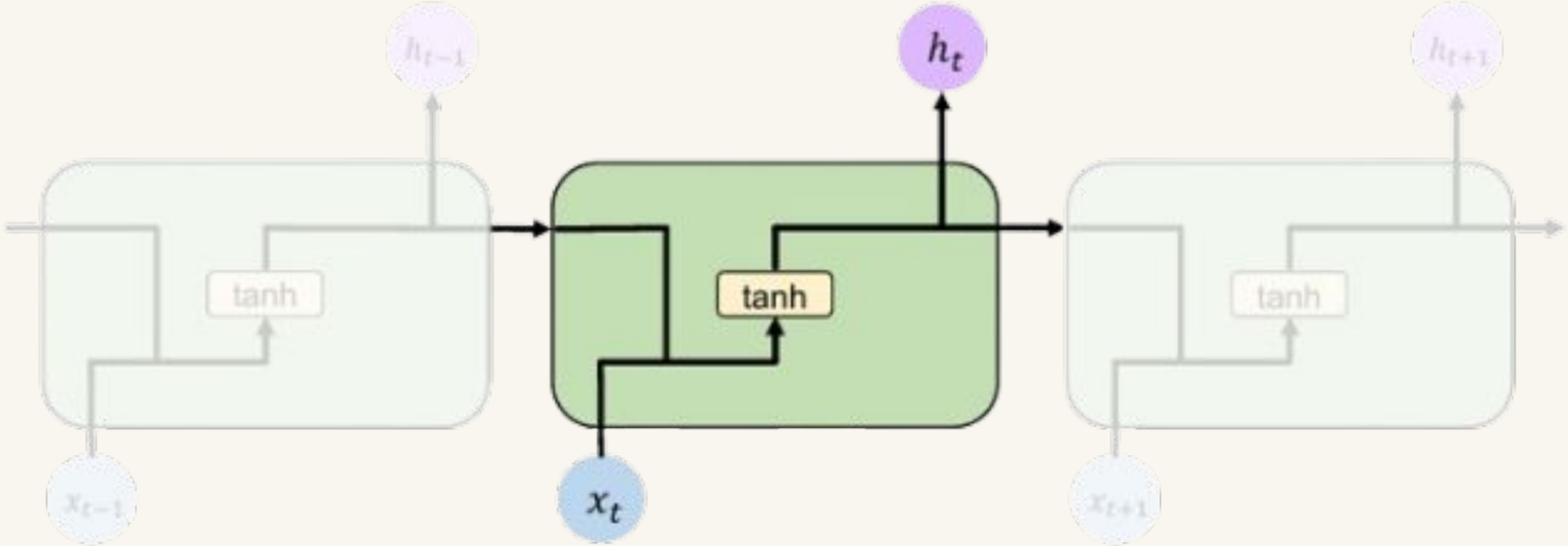


Long Short Term Memory (LSTMs) networks rely on a gated cell to track information throughout many time steps.

Long Short Term Memory (LSTM) Networks

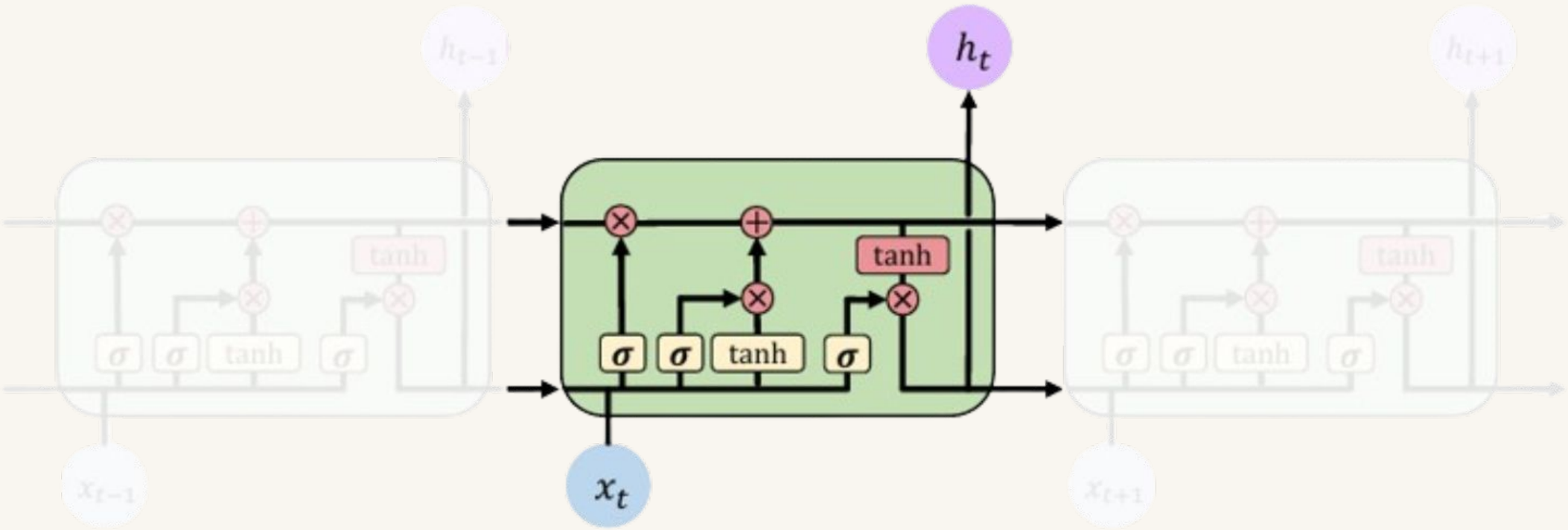
Standard RNN

In a standard RNN, repeating modules contain a simple computation node



Long Short Term Memory (LSTMs)

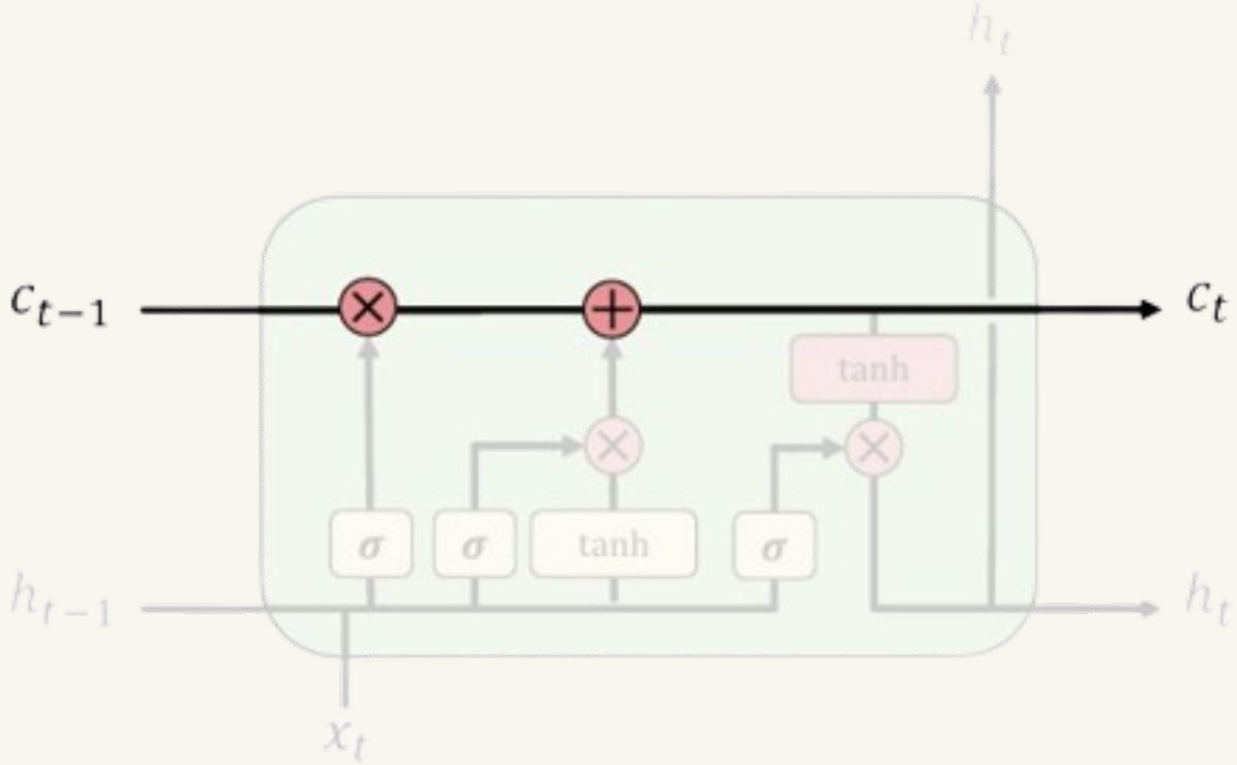
LSTM repeating modules contain interacting layers that control information flow



LSTM cells are able to track information throughout many timesteps

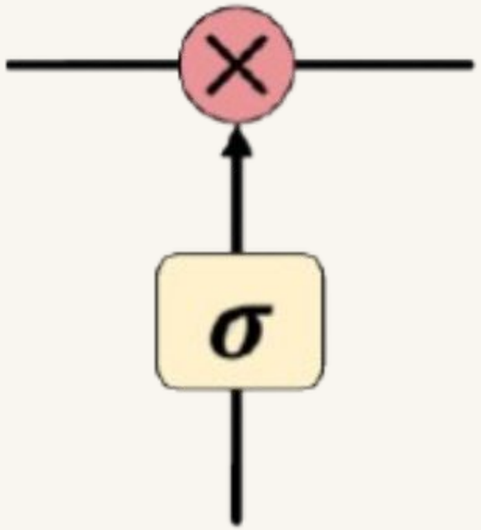
Long Short Term Memory (LSTMs)

LSTMs maintain a **cell state** c_t where it's easy for information to flow



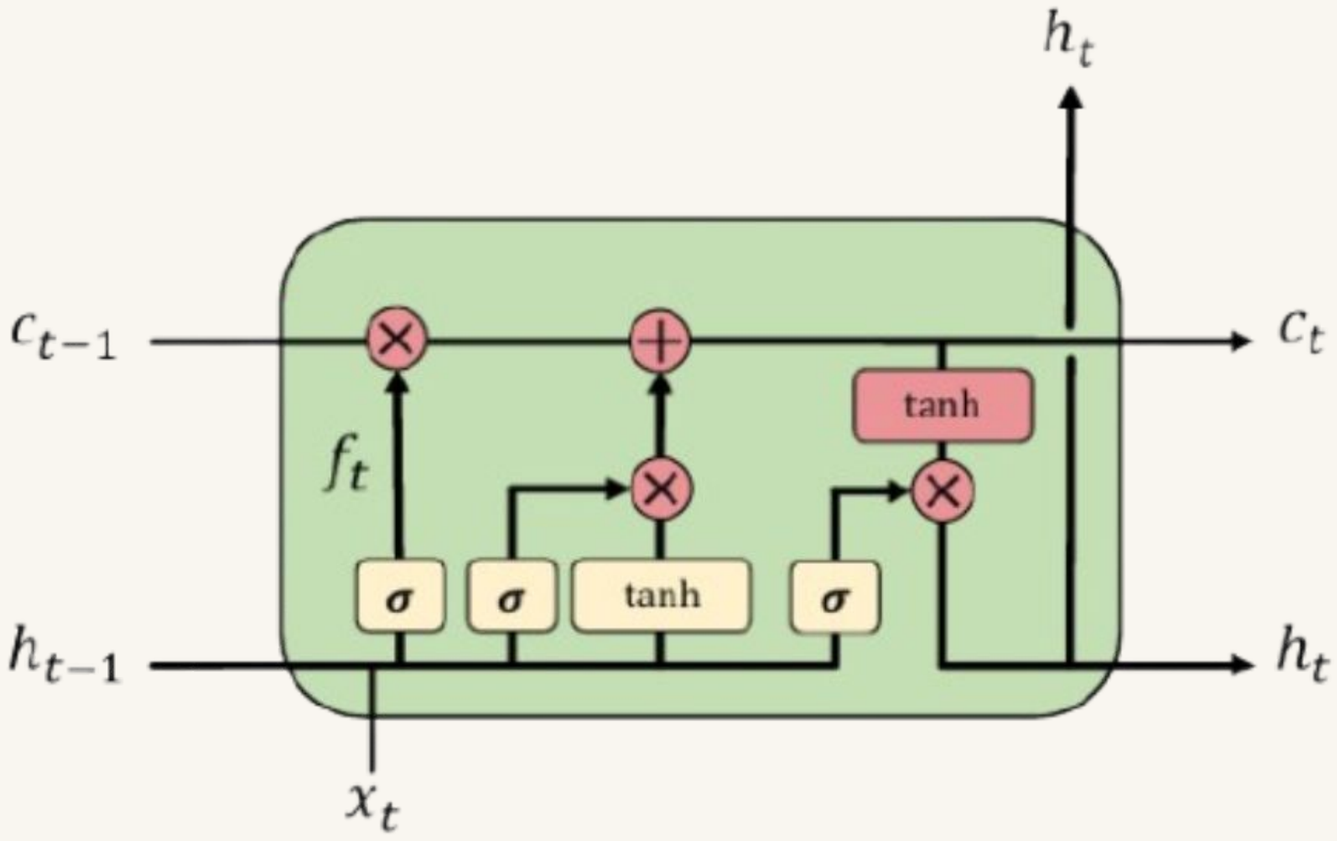
Long Short Term Memory (LSTMs)

Information is added or removed to cell state through structures called gates



Gates optionally let information flow through, via a sigmoid neural net layer and pointwise multiplication

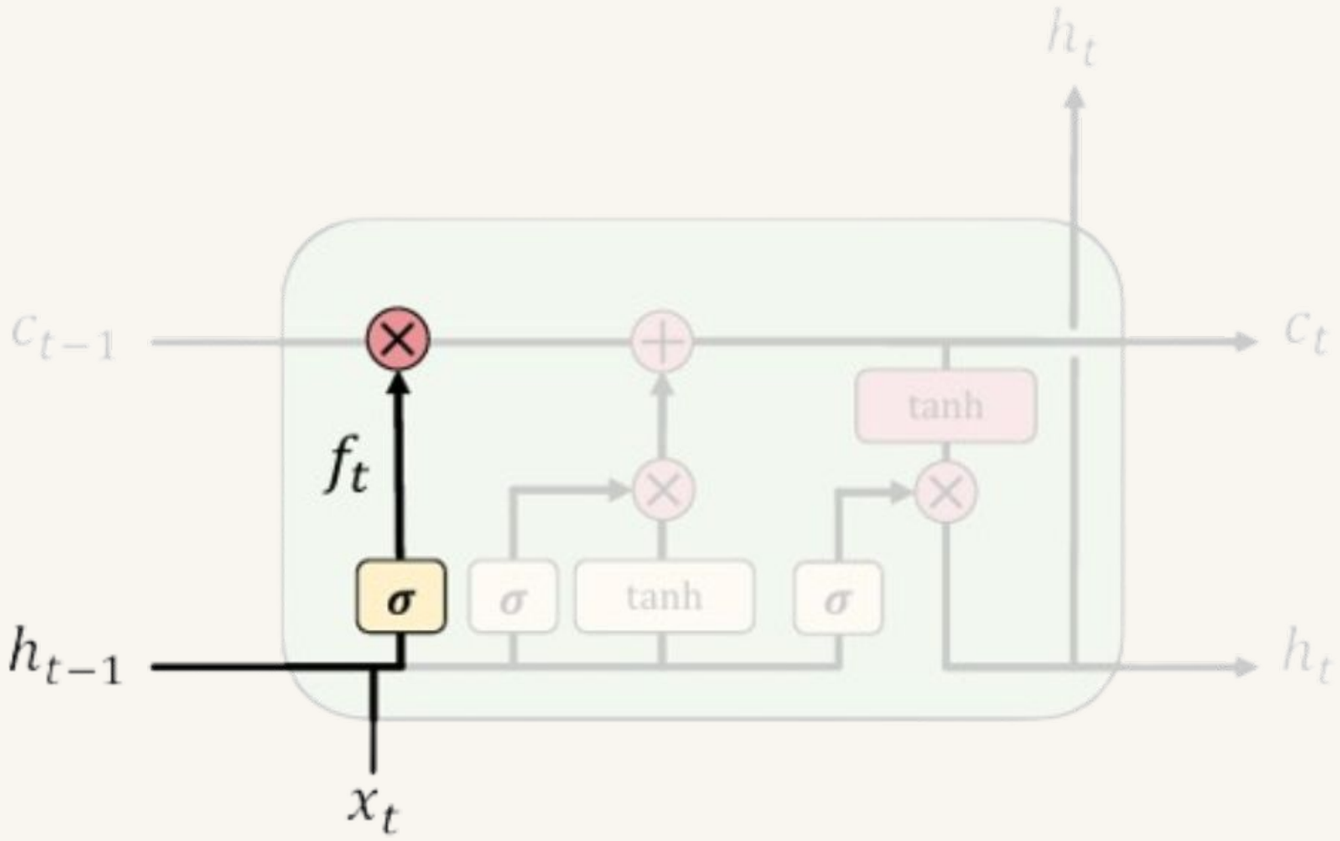
How do LSTMs work?



ct = long term memory (It carries information across long sequences without much interference.)
ht = short term memory (It focuses on what is relevant right now.)

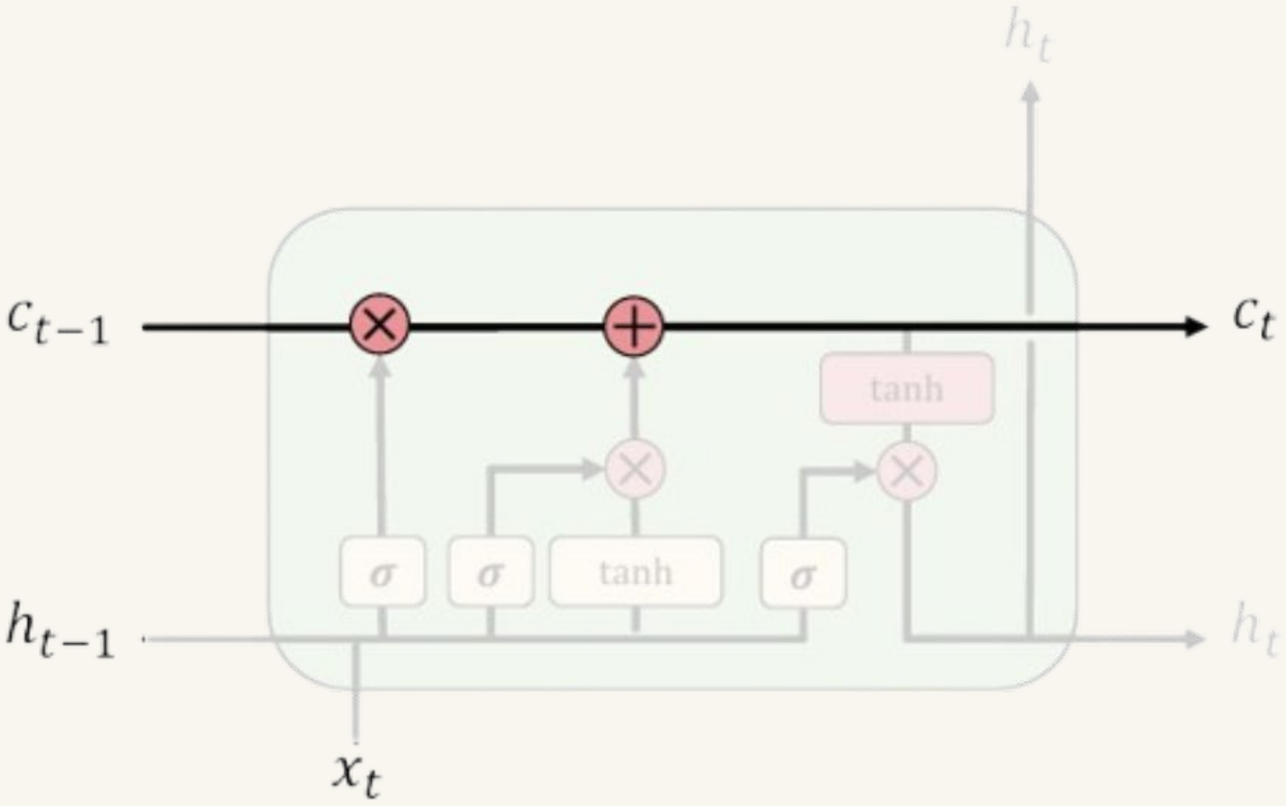
Long Short Term Memory (LSTMs)

LSTMs **forget irrelevant** parts of the previous state



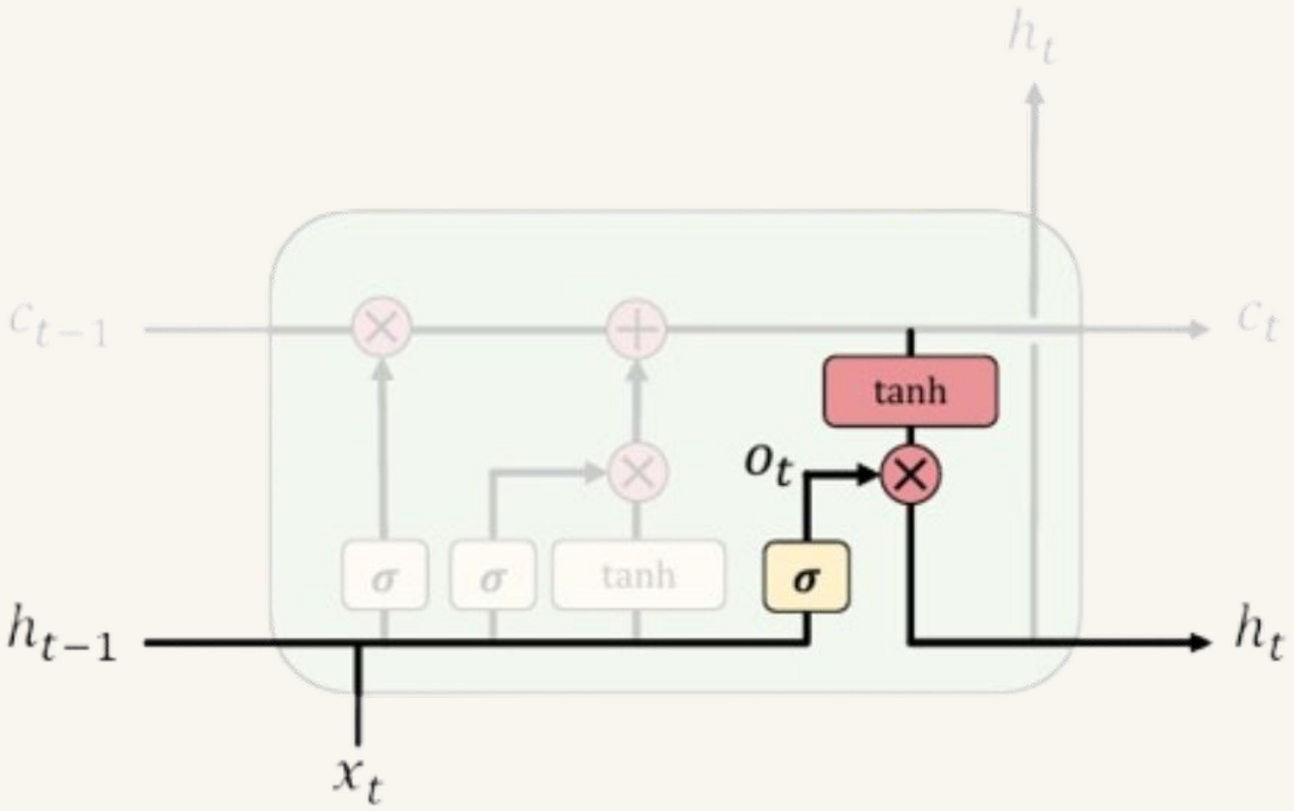
Long Short Term Memory (LSTMs)

LSTMs **selectively update** cell state values



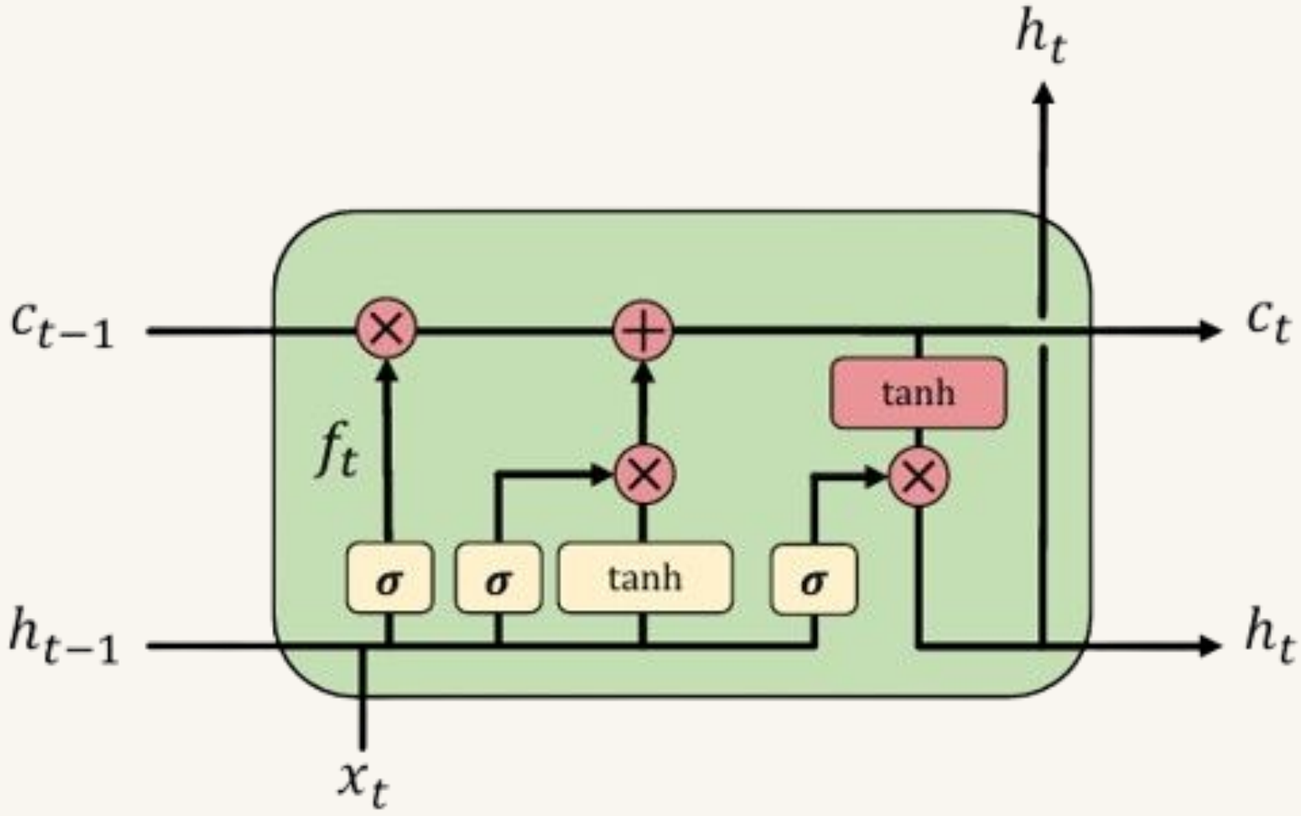
Long Short Term Memory (LSTMs)

LSTMs use an **output gate** to output certain parts of the cell stat

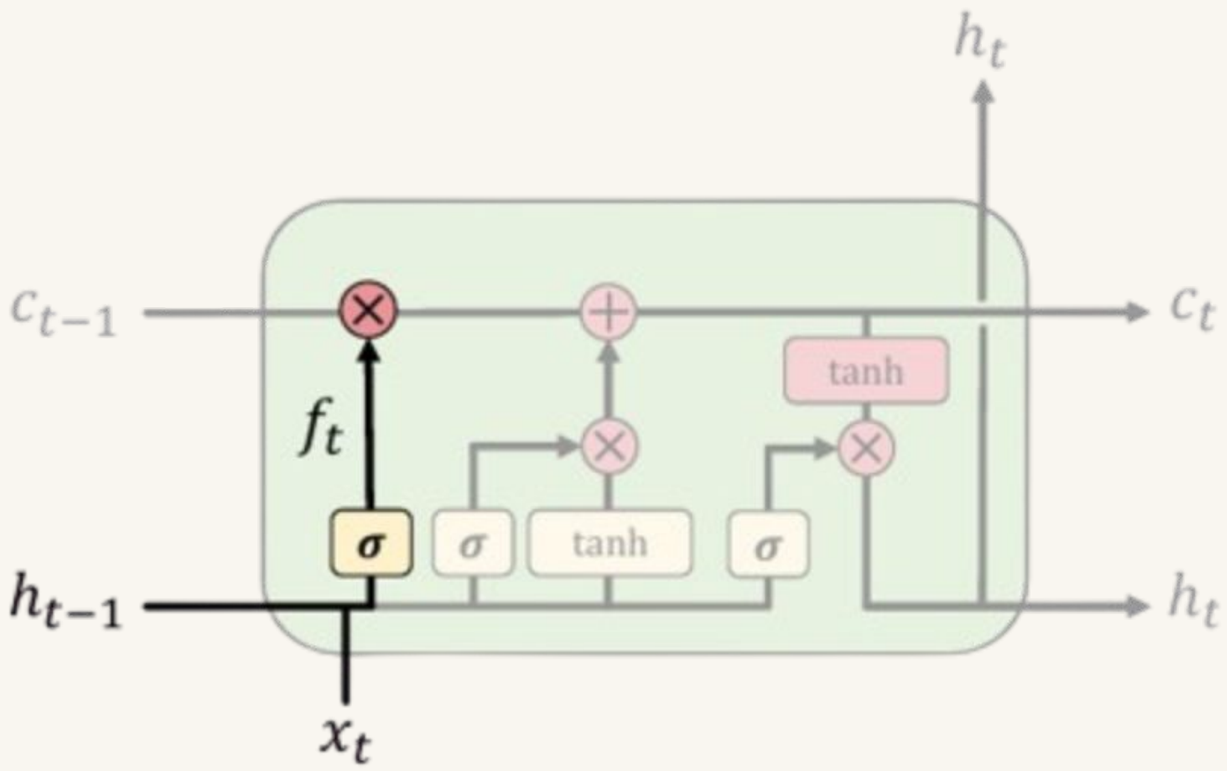


Long Short Term Memory (LSTMs)

With the help of **Forgate**, **Update** and **Output** gate LSMT works



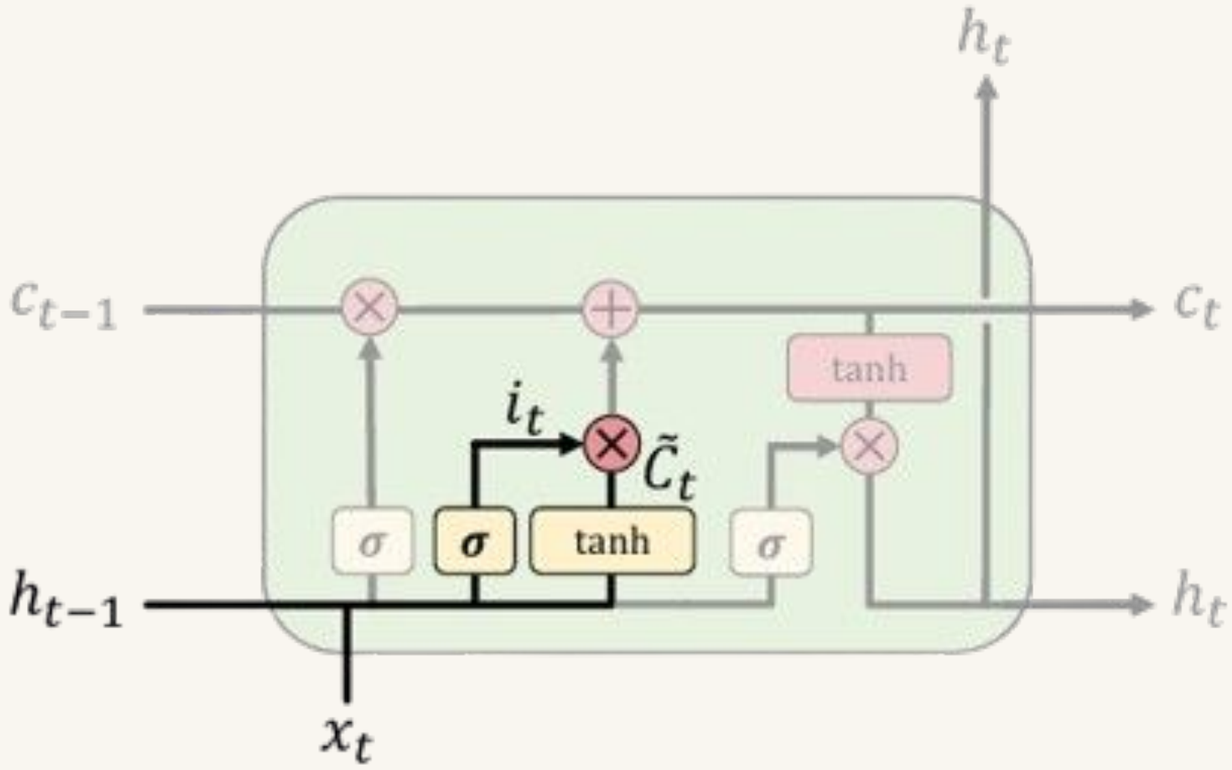
LSTMs: forget irrelevant information



$$f_t = \sigma(\boldsymbol{W}_i[\, h_{t-1}, x_t \,] + b_f)$$

- Use previous cell output and input
- Sigmoid: value 0 and 1 - "completely forget" vs. "completely keep"

LSTMs: identify new information to be stored

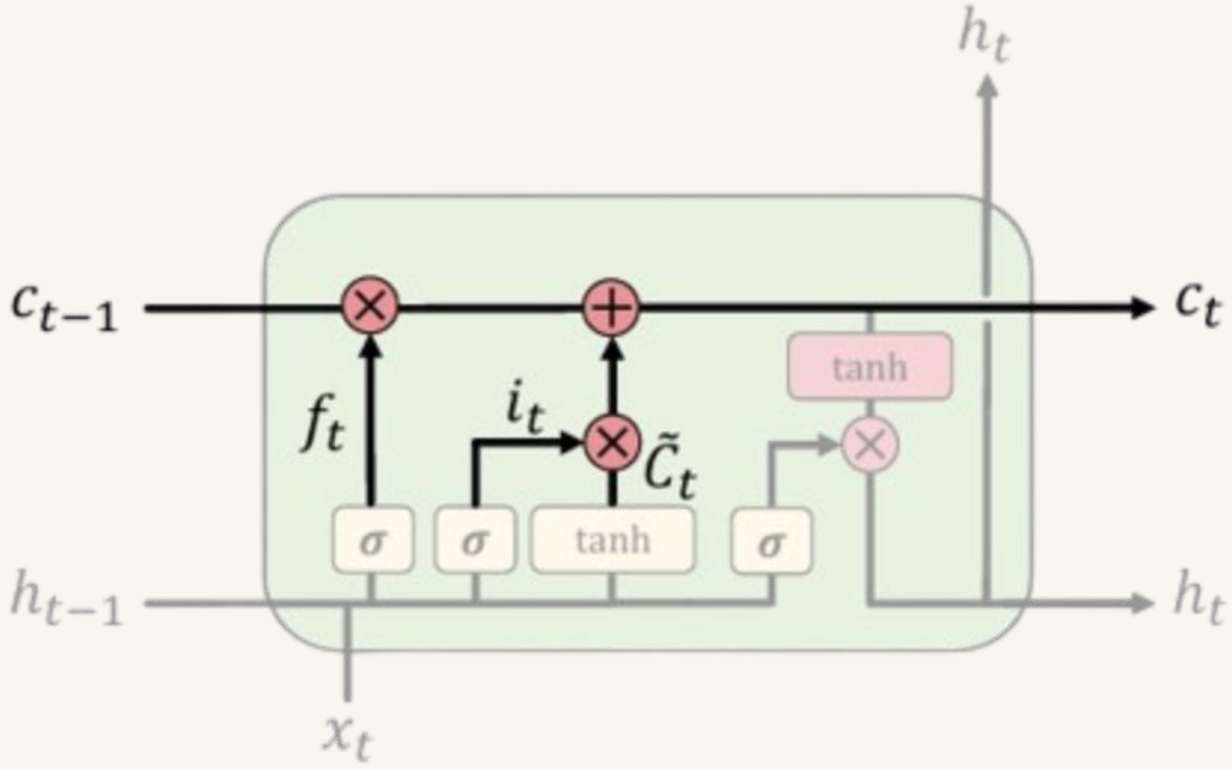


$$i_t = \sigma(\boldsymbol{W}_i[\, h_{t-1}, x_t \,] + b_i)$$

$$\tilde{C}_t = \tanh(\boldsymbol{W}_C[\, h_{t-1}, x_t \,] + b_C)$$

- Sigmoid layer: decide what values to update
- Tanh layer: generate new vector of "candidate values" that could be added to the state

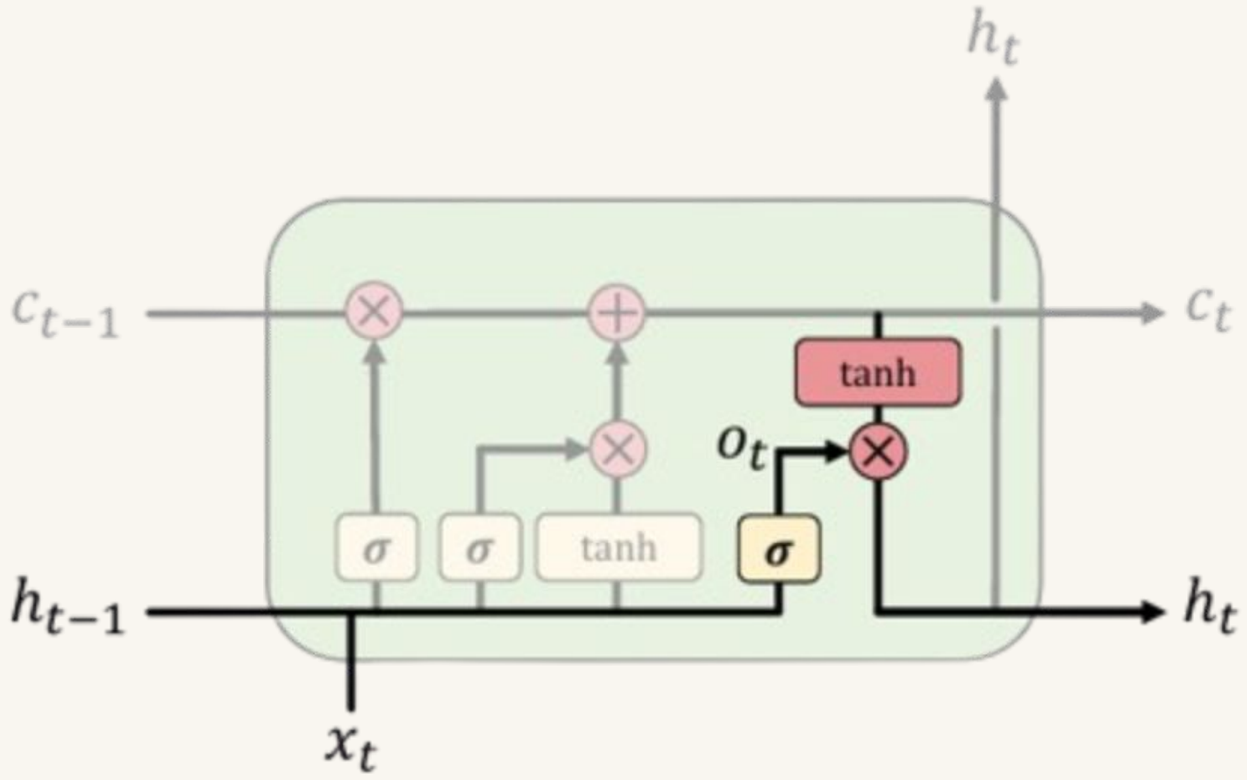
LSTMs: update cell state



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

- Apply forget operation to previous internal cell state: $f_t * C_{t-1}$
- Add new candidate values, scaled by how much we decided to update: $i_t * \tilde{C}_t$

LSTMs: output filtered version of cell state



$$o_t = \sigma(\boldsymbol{W}_o[\, h_{t-1}, x_t \,] + b_o)$$

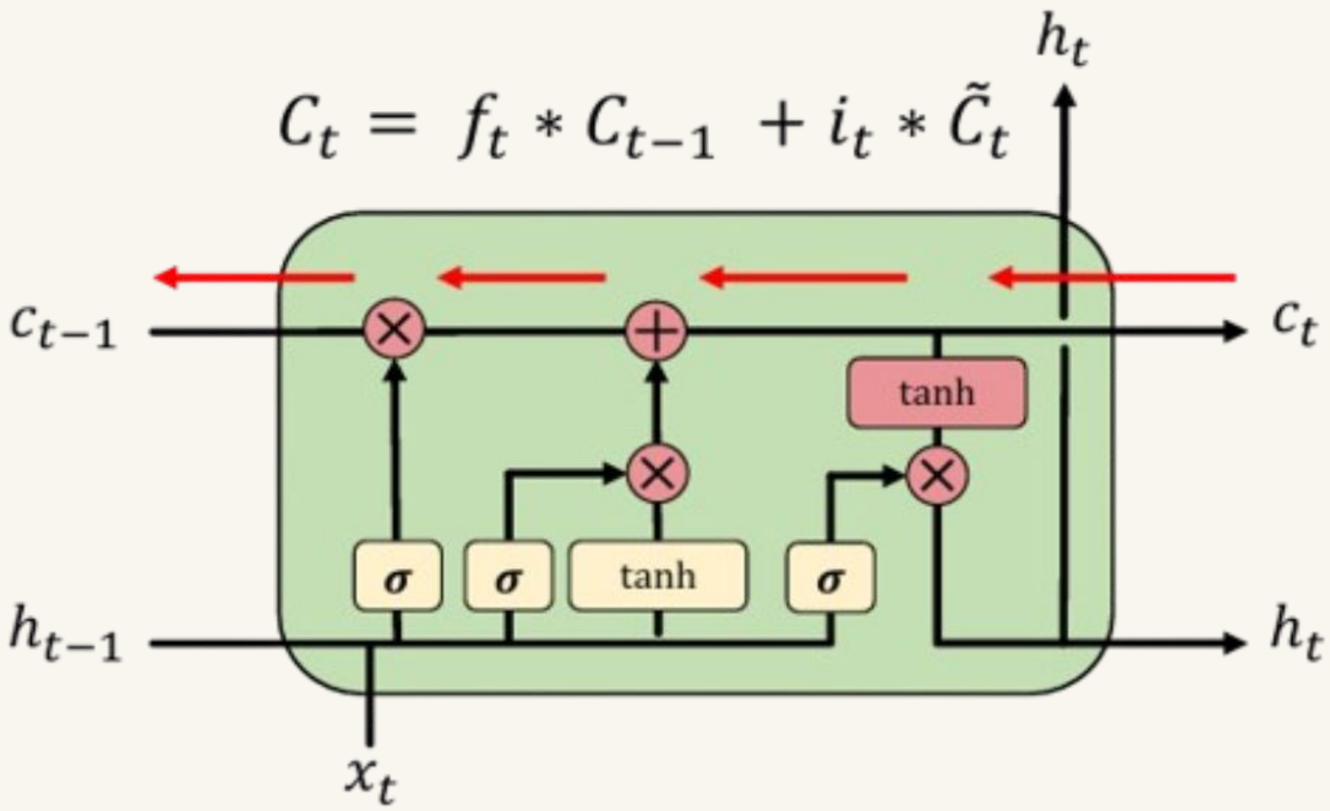
$$h_t = o_t * \tanh(C_t)$$

- Sigmoid layer: decide what parts of state to output
- Tanh layer: squash values between -1 and 1
- $o_t * \tanh(C_t)$: output filtered version of cell state

LSTM gradient flow

Backpropagation from C_t to C_t-1 requires only **element wise** multiplication .

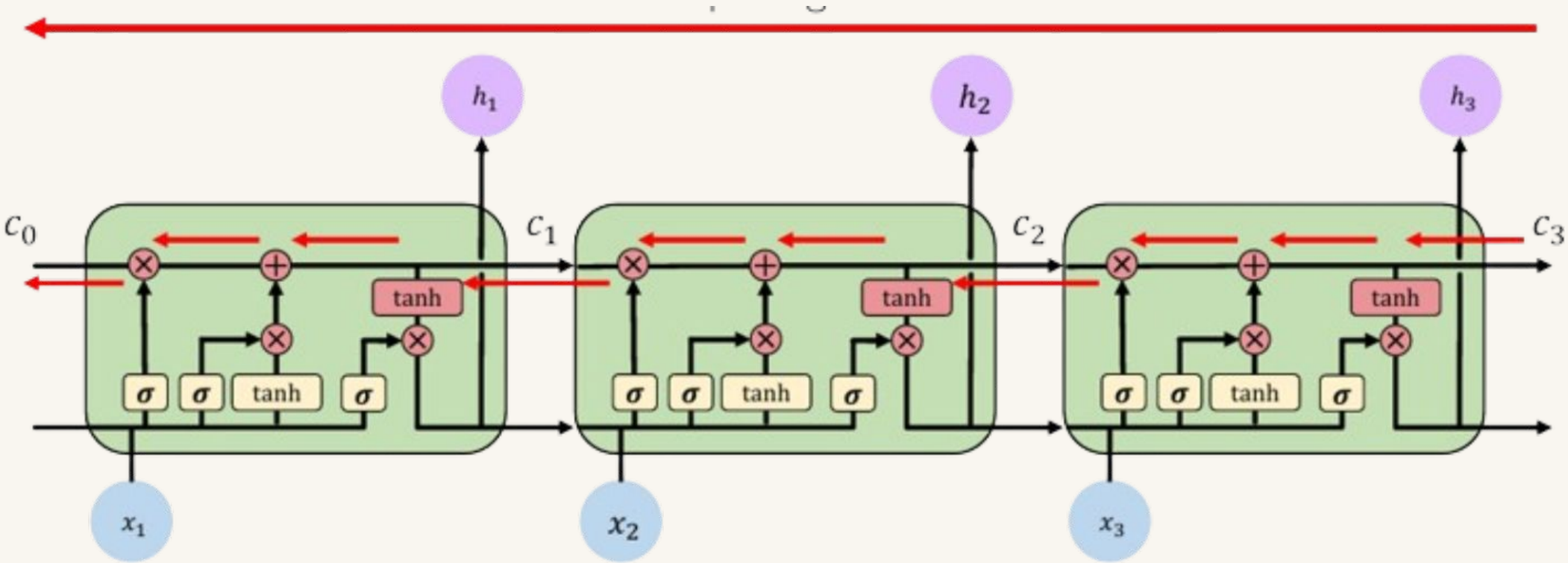
No matrix multiplication to avoid vanishing gradient problem.



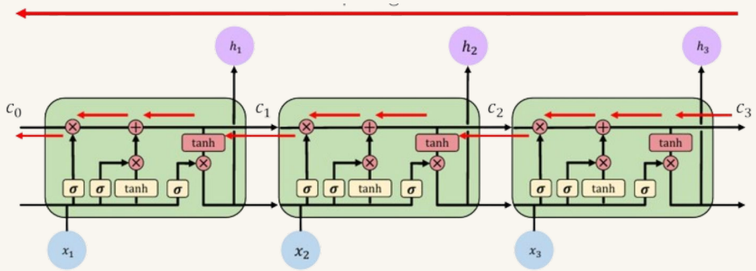
LSTM gradient flow

Backpropagation from C_t to C_t-1 requires only **element wise** multiplication .

No matrix multiplication to avoid vanishing gradient problem.



LSTM gradient flow



1. The Standard RNN

In a standard RNN, the gradient is multiplied by a weight (W) at every single step. Let's say our weight W = 0.5.

- **Step 10:** Gradient is 1.0
- **Step 9:** 1.0 x 0.5 = 0.5
- **Step 8:** 1.0 x (0.5 x 0.5) = 0.25
- **Step 1:** 1.0 x (0.5)^9 ~ 0.0019

By the time the signal reaches the first word, it is almost **zero**. The model "forgets" what happened at the start.

2. The LSTM

In an LSTM, the gradient flows through the **Forget Gate f_t**. This isn't a fixed weight; it's a value between 0 and 1 that the model chooses. If the model decides a memory is important, it sets the Forget Gate near **1.0**.

Let's say the model wants to remember something, so it sets f_t = 0.99.

- **Step 10:** Gradient is 1.0
- **Step 9:** 1.0 x 0.99 = 0.99
- **Step 8:** 1.0 x (0.99 x 0.99) = 0.98
- **Step 1:** 1.0 x (0.99)^9 ~ 0.91

The Result

- **RNN Signal:** 0.0019 (The signal vanished).
- **LSTM Signal:** 0.91 (The signal is still strong).

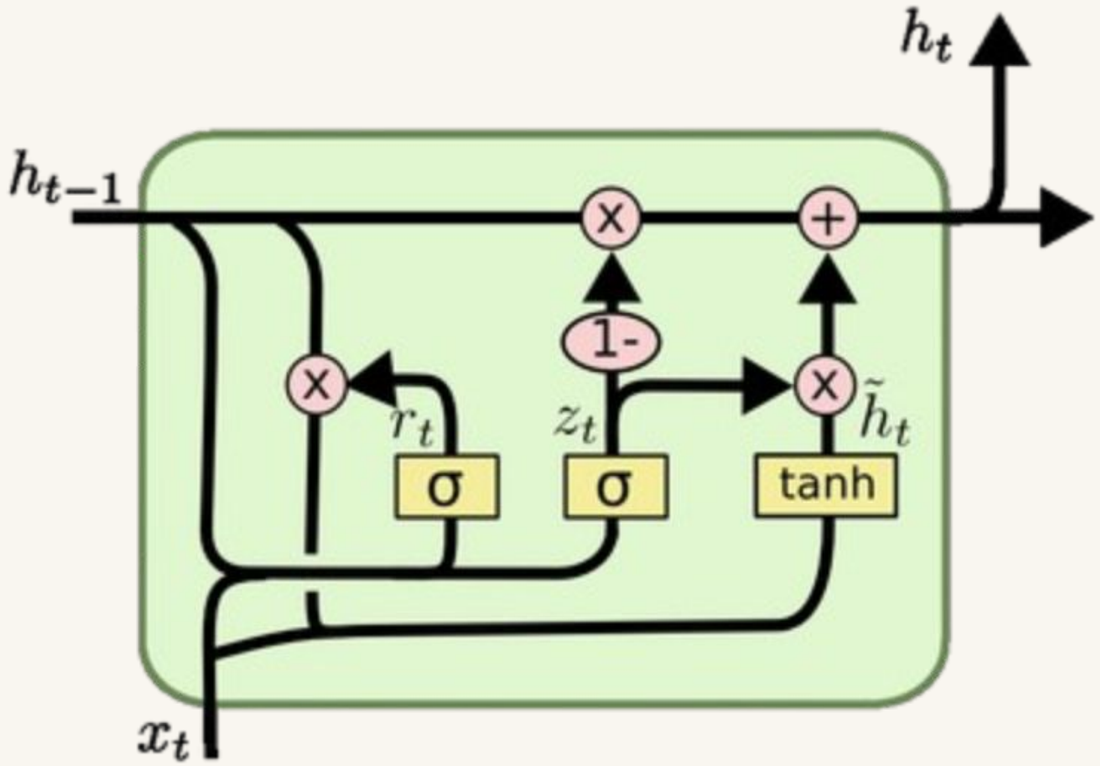
Because the LSTM uses **addition** and **element-wise multiplication**, it acts like a **volume knob**. If the knob is turned up 1.0, the information flows through the "highway" perfectly preserved, no matter how long the sentence is.

LSTMs: key points

1. Maintain a **separate cell state** from what is outputted.
2. Use **gates** to control the **flow of information**
   a. **Forget gate gets rid of irrelevant information**
   b. **Selectively update cell state**
   c. **Output gate returns a filtered version of the cell state**
3. Backpropagation from C_t to C_t-1 doesn't require matrix multiplication: uninterrupted gradient flow.
4. <u>All the weights in the LSTM **do get updated**, including those involving hidden state and input. In LSTM the term **c_t** allows gradients to pass almost unchanged. Hence, gradient doesnot vanishes anymore</u>

Gated Recurrent Unit (GRU)

Gated Recurrent Unit (GRU)



$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$$

$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$

$$\tilde{h}_t = \tanh\left(W \quad [r_t * h_{t-1}, x_t]\right)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

LSTM VS GRU

Both LSTM and GRU are variants of RNNs, equipped with gating mechanisms that regulate the flow of information and reduce the impact of vanishing gradients.

1. LSTM includes **three gates**: input, forget, and output, along with a memory cell that allows it to **retain long-term dependencies more effectively.**
2. GRU, on the other hand, simplifies this structure using only **two gates**: update and reset. While less complex, it performs comparably in many tasks and is **faster to train.**

Thank You